Applied Machine Learning

Hyperparameter Tuning
Nested Resampling & Test Set Validation

Learning goals

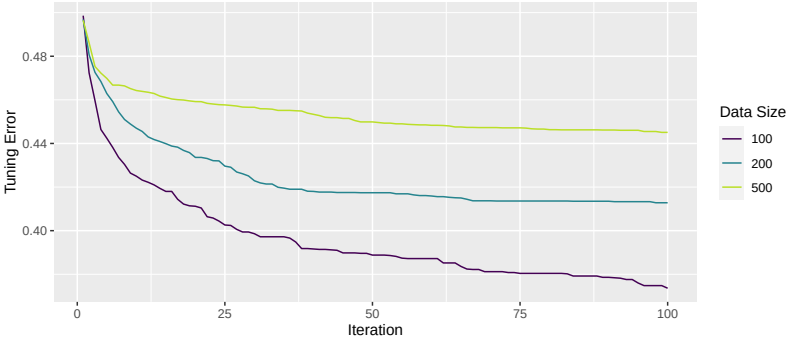
- Understanding the untouched test set principle
- Nested resampling

The Untouched Test Set Principle

INSTRUCTIVE AND PROBLEMATIC EXAMPLE

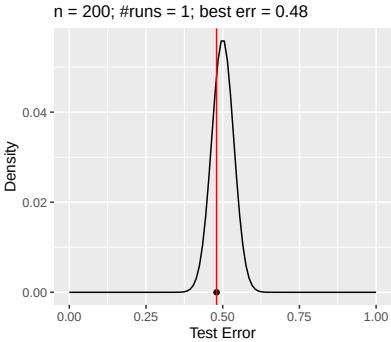
- Assume a binary classification problem with equal class sizes.
- Assume a learner with hyperparameter λ.
- Here, the learner is a (nonsense) feature-independent classifier, where λ has no effect. The learner simply predicts random labels with equal probability.
- Of course, its true generalization error is 50%.
- A cross-validation of the learner (with any fixed λ) will easily show this (given that the partitioned data set for CV is not too small).
- Now let's "tune" it, by trying out 100 different λ values.
- We repeat this experiment 50 times and average results.

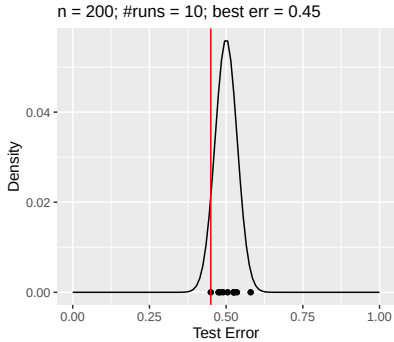
INSTRUCTIVE AND PROBLEMATIC EXAMPLE / 2



- Plotted is the best “tuning error” (i.e. the performance of the model with fixed λ as evaluated by the cross-validation) after k tuning iterations.
- We have performed the experiment for different sizes of learning data that were cross-validated.

INSTRUCTIVE AND PROBLEMATIC EXAMPLE / 3





- For 1 experiment, the CV score will be nearly 0.5, as expected
- We basically sample from a (rescaled) binomial distribution when we calculate error rates
- And multiple experiment scores are also nicely arranged around the expected mean 0.5

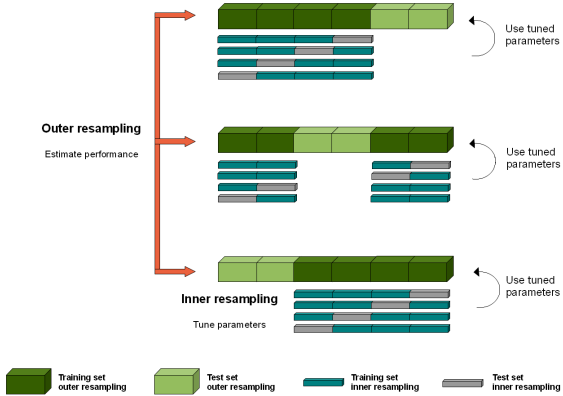
NESTED RESAMPLING

Just like we can generalize hold-out splitting to resampling to get more reliable estimates of the predictive performance, we can generalize the training/validation/test approach to **nested resampling**.

This results in two nested resampling loops, i.e., resampling strategies for both tuning and outer evaluation.

NESTED RESAMPLING / 2

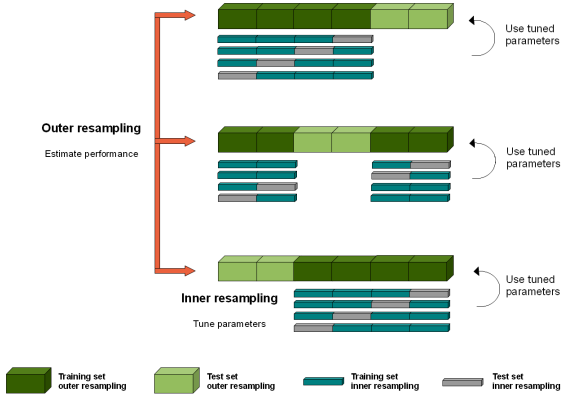
Assume we want to tune over a set of candidate HP configurations λ_i; $i = 1, \ldots$ with 4-fold CV in the inner resampling and 3-fold CV in the outer loop. The outer loop is visualized as the light green and dark green parts.



NESTED RESAMPLING / 3

In each iteration of the outer loop we:

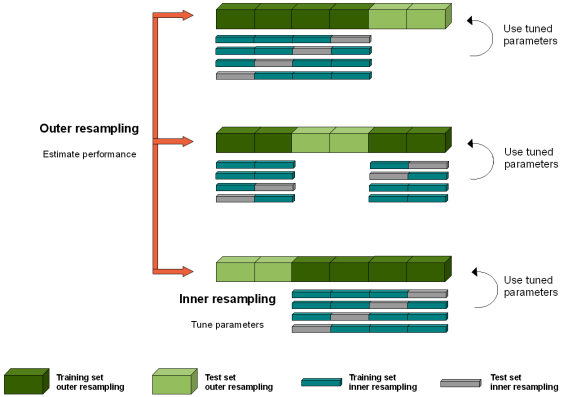
- Split off the light green testing data
- Run the tuner on the dark green part of the data, e.g., evaluate each λ_i through fourfold CV on the dark green part



NESTED RESAMPLING / 4

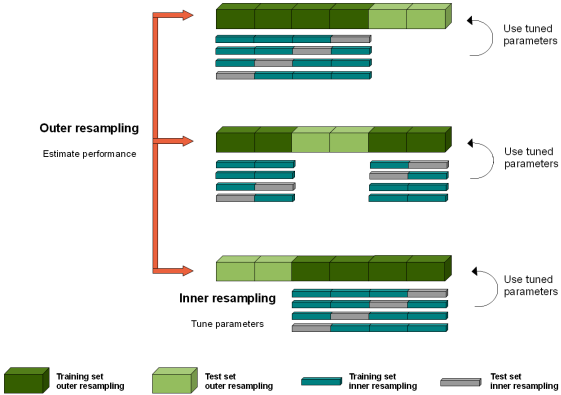
In each iteration of the outer loop we:

- Return the winning λ^* that performed best on the grey inner test sets
- Re-train the model on the full outer dark green train set
- Evaluate it on the outer light green test set



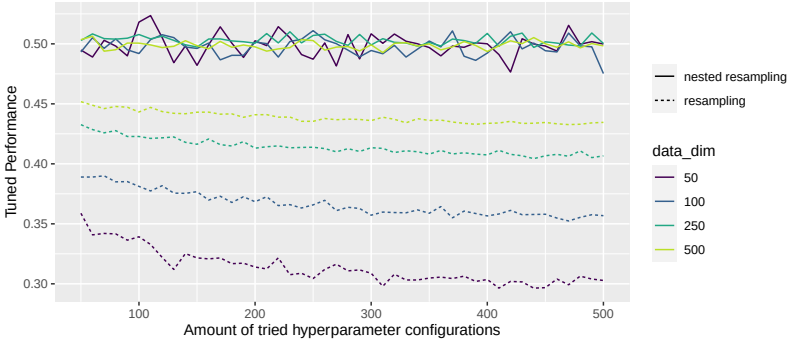
NESTED RESAMPLING / 5

The error estimates on the outer samples (light green) are unbiased because this data was strictly excluded from the model-building process of the model that was tested on.



NESTED RESAMPLING - INSTRUCTIVE EXAMPLE

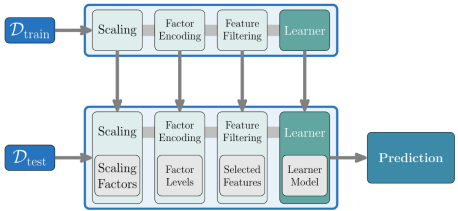
Taking again a look at the motivating example and adding a nested resampling outer loop, we get the expected behavior:

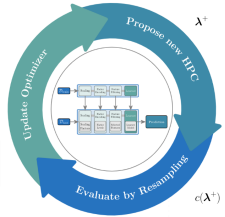


Pipelines

PIPELINES AND AUTOML

- ML typically has several data transformation steps before model fit
- If steps are in succession, data flows through sequential pipeline
- NB: Each node has a train and predict step and learns params
- And usually has HPs

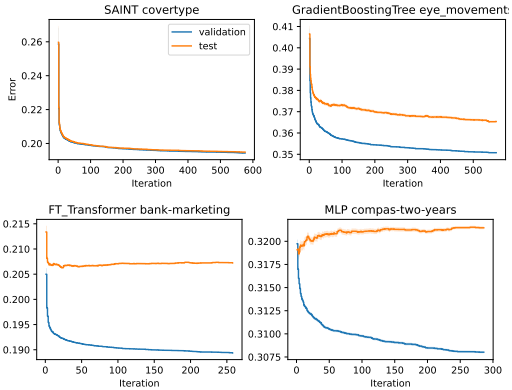




Pipelines are required to embed full model building into CV to avoid overfitting and biased evaluation!

Practical Considerations

VALID-TEST CURVES IN HPO



- **Ideal (top-left):** validation and test improve together.
- **Meta-overfitting (top-right):** validation too optimistic, test still improves.
- **Overtuning (bottom-left):** validation keeps improving, test deteriorates.
- **Severe overtuning (bottom-right):** later incumbents can be worse than much earlier ones on test data.

OVERTUNING

Let $\boldsymbol{\lambda}_t^*$ be the incumbent (best-performing configuration) at step t, i.e. the configuration with the best validation score seen so far.

Meta-overfitting: validation looks better than test for the current incumbent

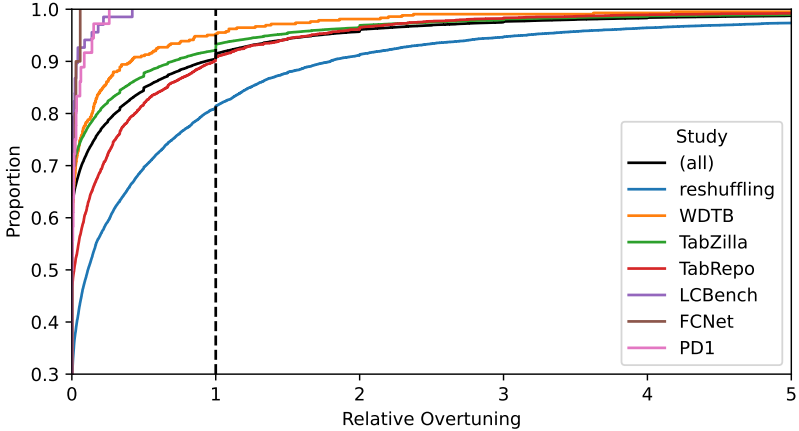
$$\mathrm{of}_t = \widehat{\mathrm{test}}(\boldsymbol{\lambda}_t^*) - \widehat{\mathrm{val}}(\boldsymbol{\lambda}_t^*)$$

Overtuning: the current incumbent is worse on test than the best incumbent seen so far

$$\mathrm{ot}_t = \widehat{\mathrm{test}}(\boldsymbol{\lambda}_t^*) - \min_{t' \leq t} \widehat{\mathrm{test}}(\boldsymbol{\lambda}_{t'}^*)$$

- Meta-overfitting is a **gap** between validation and test.
- Overtuning is a **selection error**: we keep improving validation but select a worse model.
- Nonzero meta-overfitting is possible without overtuning.
- In HPO, overtuning is usually the more important practical problem.

OVERTUNING

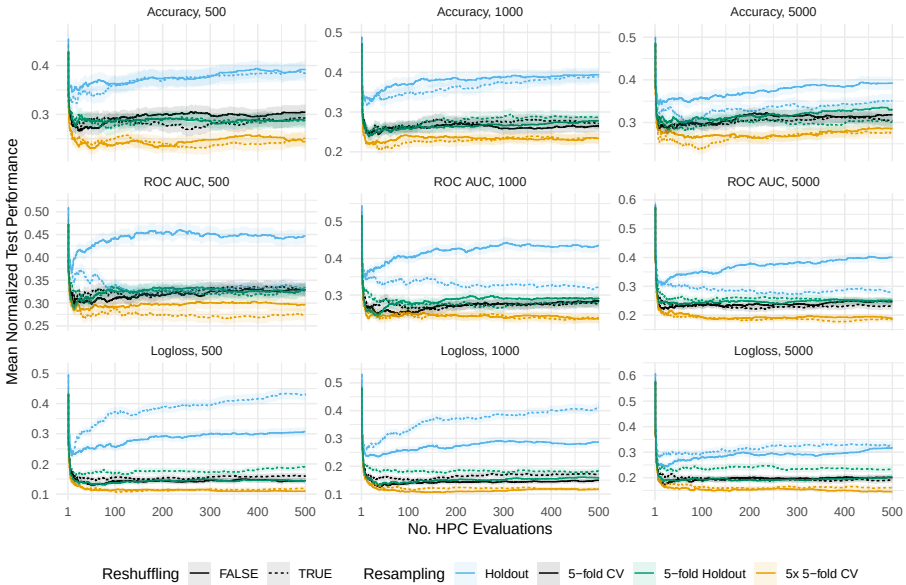

1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
Proportion
0
1
2
3
4
5
Relative Overtuning
Study
(all)
reshuffling
WDTB
TabZilla
TabRepo
LCBench
FCNet
PD1

REMEDIES AGAINST OVERTUNING

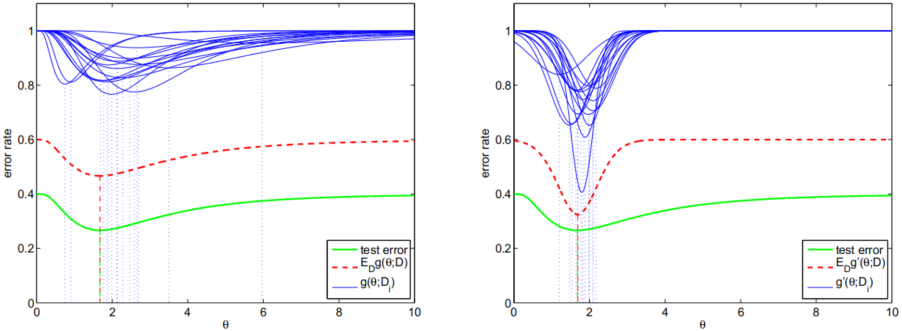
- **More data / larger validation sets:** reduces noise in validation estimates.
- **Repeated CV or reshuffling splits:** stabilizes the HPO objective.
- **Be skeptical about tiny late gains:** they may be pure validation noise.
- **Keep strong baselines:** defaults or early incumbents can already generalize best.

Main message from the paper: overtuning is especially relevant in the **small-data regime**.

VISUALIZATION



VARIANCE IN RESAMPLING



Recap

IMPORTANCE OF CV ON KAGGLE

- Understand the data - After you download the data, start exploring features. Look at data types. Check variable classes. Create some univariate - bivariate plots to understand the nature of variables.
- Understand the metric to optimize - Every problem comes with a unique evaluation metric. It's imperative for you to understand it, specially how does it change with target variable.
- Decide cross validation strategy - To avoid overfitting, make sure you've set up a cross validation strategy in early stages. **A nice CV strategy will help you get reliable score on leaderboard**.
- Start hyper parameter tuning

Source: KazAnova, former Kaggle Champion at hackerearth.com tutorial

TAKE-HOME MESSAGES

- Conduct correct cross-validation
- Nested-Resampling for performance estimation
- Use advanced and robust HPO
- Include your preprocessing in the splitting